API Quality Studio

User manual

What API Quality Studio is

API Quality Studio tests your APIs on your own computer. You give it an API description (a Postman collection, an OpenAPI or Swagger file, a SOAP WSDL, a GraphQL endpoint or even a written API document) and valid test data. It then creates valid, negative, boundary and security tests, runs them against the environment you name, and produces reports: HTML, Excel, PDF and a vulnerability (VAPT) report.

Everything runs locally. There is no cloud service and no online key. Your API descriptions, data and credentials never leave your computer. An optional local language model can make the built-in assistant sound more natural, but the product works fully without it.

Quick start in five steps

- Press Try the demo on Home to see the whole flow on a safe public sandbox in two minutes.
- For your own API press + New project. Choose your file (or type a GraphQL address), enter the application URL and press Create.
- Open Overview and press Set credentials if the API needs a sign-in. Use a test account.
- Fill the Test data workbook with valid data that exists in your test environment and upload it.
- Press Run full assessment, confirm that you are authorised, and watch the Live run page. Open Reports when it ends.

Using Studio on another computer (for example a laptop)

- Copy the package zip (APIQualityStudio-1.0.0.zip) to the other computer and unzip it anywhere. No installation and no Java are needed.
- Double-click Start API Quality Studio.bat. Studio opens in the browser. Projects and reports are kept in the folder APIQualityStudio-data in your user folder.
- To bring your projects along, copy the projects folder of the old computer into that APIQualityStudio-data folder.
- Install your licence (left menu, Licence) or copy license.json into the folder .api-quality-studio in your user folder. Without it the product runs as a 30-day trial.
- Credentials are never copied with a project. Enter them again under Set credentials on the new computer.

Key ideas

Projects

A project holds one application: its APIs, tests, data links, reports and history. Everything is stored as readable files under the projects folder. Switch projects in the left menu.

Tests and their states

Every test has a state. DRAFT is new and does not run. REVIEWED means a person checked it. APPROVED may run. ACTIVE is part of the regular suite. Tests that Studio generates always start as DRAFT, except when you press Run full assessment, which approves what can safely run after you confirm your authorisation.

Outcomes

- PASS: the API behaved as expected.
- FAIL: a finding. The API behaved differently from the expectation or the rule pack.
- BLOCKED: the test was not sent because something it needs is missing or failed (for example a login or a value). This hides coverage, but it is not an API failure.
- NOT_EXECUTED: the test is not eligible to run (draft, switched off, unsupported body).
- The pass rate is passed divided by passed plus failed. Blocked and not-executed tests are shown separately.

Positive first

A negative test is only judged when the valid request of the same API passed. Otherwise it is BLOCKED with the reason, so a wrong URL or wrong data never produces hundreds of misleading results.

Bringing in your API

Supported sources

- Postman collection v2.1 (JSON) and OpenAPI 3 or Swagger 2 (JSON).
- SOAP WSDL 1.1 (SOAP 1.1 and 1.2). Each operation becomes a request with a sample envelope.
- GraphQL. Type the endpoint address and Studio reads the schema by introspection, or import an introspection result file.
- gRPC. Type the server address (grpc://host:50051 or grpcs://host:443). Studio discovers the services and unary methods through gRPC reflection and sends JSON requests. Checks that change a field's type do not apply to gRPC and are not generated. Streaming methods are reported and skipped.
- WebSocket. Type a wss:// address, or use Add WebSocket request on the APIs page. Studio connects, sends your message, waits for the answer and checks what it must contain. A refused connection (for example 401) and an error close by the server are judged like HTTP answers.
- API documents in Word, PDF, Markdown or text. Studio reads lines such as POST /customers, the JSON block after them, headings as modules and a Base URL line. Endpoints without a visible body are reported, never invented.

The application URL

Enter the address of the API, not the address of the web page. The API address is the part that comes before your API paths, for example https://host/api. On the Overview, Verify my APIs asks the server whether your paths exist there (OPTIONS requests and read-only GETs, nothing is executed). A server that answers every path alike is a website; Studio says so and stops before sending anything. Find API address tries the usual addresses and the base URLs defined inside your collection.

Credentials

Open Overview and use Set credentials. Choose a login API (user name and password, Studio signs in once and sends the token), a bearer token, HTTP Basic or an API key. Values stay in memory for the session, or encrypted on this computer if you tick Remember. Passwords and tokens are never saved in project files, the test-data workbook, the history or any report. If a value is missing the tests that need it are blocked with the reason. A web application with a one-time code (OTP) cannot be signed in by Studio; use Record from browser and log in yourself.

Use a test account. Login tests send only a few wrong passwords on purpose, and Studio caps them, but a real person's account could still be locked.

Test data

Download the workbook from Test data. It is created from the valid request of every API: bodies and headers from your file, the expected 2xx status and the values that are extracted or set. Passwords appear as references, not values. Change what is invalid for your environment, then upload it. Rows with Run=N are switched off and new rows become new APIs. Uploading the file unchanged changes nothing.

Fill empty sample fields finds fields that are empty or still say "string" and proposes clearly artificial values (an example.test e-mail, a sample mobile number). You review and apply. Passwords, tokens, user names and ids from other calls are never touched.

Making tests

Generate negative and security tests

Studio derives negative and security tests from your valid tests: missing fields, wrong types, boundaries, injection markers, no token, tampered token, other methods, oversized bodies and more. One thing changes per test. Login endpoints get one wrong-password attempt only. Create, update and delete APIs get security checks only unless you opt in to data tests.

Data flow (chaining)

Open Data flow. Studio finds values one API produces and another needs, such as an id returned by POST /pet and used in GET /pet/{petId}. Review the detected links, add them, and add your own when needed. A run then follows the order: set up (create), test, clean up (delete what was created). Remove a link and the original sample value returns.

Business rules

Upload a requirement document under Documents, then open Business rules. Sentences such as "BR-001 POST /pet: the name is required and at most 20 characters" become proposals: required field, length, allowed values, numeric limits, response must contain a field, response time and success status. Each proposal shows its source sentence. Anything unclear, such as which API or which field, is shown for you to decide. Nothing is created until you tick it. Rules that are too vague are listed as not testable, never guessed. New tests start as DRAFT.

Contract checking and spec drift

When the project comes from an OpenAPI or Swagger file, every valid response is checked against its response contract. Choose on the APIs page whether a deviation is ignored, passes with a remark, or fails the test. Compare with a newer spec shows what was added, removed or changed. Applying it is safe: removed APIs have their tests parked as DRAFT, changed APIs are tagged spec-changed, new APIs arrive as DRAFT, and nothing is deleted.

Mock server

On the APIs page you can start a local mock server that answers your APIs from their contract while the real backend is not ready. You can add latency and a percentage of failures. It listens on this computer only.

Test ideas

Open Test ideas to get tests beyond the standard set. Built-in ideas use the meaning of each field (an e-mail without @, a mobile number with letters, a negative amount, a PAN with the wrong pattern). Tick the ones you want and add them as DRAFT tests. With the optional AI engine switched on, a language model can propose more; its ideas are checked (unknown fields and unsupported changes are dropped) and still wait for your approval.

The AI engine (optional)

The AI engine is off until you switch it on in the left menu under AI engine. It uses Ollama, on this computer or on another machine (type its address). Gentle mode keeps requests small and sends one at a time so a modest PC stays responsive. Studio works fully without it. The model only proposes ideas and wording; it never decides a test result.

Explain a failure and repair after a spec change

In a test result, Explain in plain English shows what happened, the likely cause and what to do (with a short model wording when the AI engine is on). After Compare with a newer spec, Studio proposes repairs to your valid requests (added or removed fields, changed types, new required parameters); you tick what may change.

Export as code

Automation and export, Export the tests as code gives Karate feature files or a Rest Assured and TestNG project for developers. Secrets are read from environment variables, never written.

Running

Run full assessment

This generates negative and security tests, approves what can run, sends the requests to the target you confirm, and builds every report. It asks for your authorisation, asks again for production-looking addresses, and shows how many requests will be sent. It stops before sending anything when the address is not the API or credentials are missing.

Run approved

Tests, Run approved repeats only Approved and Active tests. Select rows to run just those.

Live run

The Live run page opens when a run starts. It shows each request as it is sent with the time, outcome, HTTP status, milliseconds, API and test, plus counters, pass rate and speed. Filter by failed, blocked or passed. Click a line for the request and the response with secrets masked. You can cancel the run, or press Pause to stop after the API that is running finishes and Resume to continue. If a test needs a one-time code (OTP) or a CAPTCHA, the run waits and a box asks you to type the value; if you enter nothing the test is reported as blocked.

Load test

Open Load test to measure behaviour under many users. Each virtual user signs in on its own and repeats the valid requests you tick. You set users (up to 100), duration (up to 10 minutes), ramp-up and think time, and limits for the 95th percentile latency and the error rate. Results show throughput, latency percentiles and errors per API, with a

report. Read-only requests are the default. Studio stops by itself when more than half of the recent requests fail. Use a test environment only, never a system that real customers depend on.

Start fresh

Overview or Reports, Clear old records moves old run results and reports to reports/_archive and can remove the generated negative tests. APIs, data, credentials and history stay.

Reports and defects

Every run creates an HTML report (module-wise, searchable), an Excel workbook, a PDF summary and a VAPT report in HTML and PDF. Open them under Reports. The pass-rate change against the previous run is shown. Defects for your tracker exports the failures of the latest run as a CSV file for Jira or Azure DevOps. They are grouped per API and failure type, with redacted evidence. Nothing is sent to either system.

History keeps an audit trail of every change and run with user and time. Entries are chained by hash, so an edited or removed line is detected. Verify integrity checks the chain and you can export the trail as CSV or JSON.

Automation

Scheduled runs

Automation, Add a schedule. Studio runs the approved tests daily, weekly or every few minutes while it is open and records the result in History. A scheduled run is skipped, with the reason, when credentials are missing or the address is not the API.

Command line and build pipelines

The command line runs without the window. Example: studio-cli run MyProject --min-pass-rate 90 --junit build/studio-junit.xml. It sets the exit code for your pipeline: 0 passed, 1 findings or pass rate below the gate, 2 usage error, 3 target not reachable or not the API, 4 credentials missing. Secrets are read from environment variables. studio-cli load runs a load test with limits. studio-cli export-ci writes ready files for Jenkins, GitHub Actions, Azure DevOps and the Windows Task Scheduler.

Team mode

Studio normally serves one person on this computer. For a team, an administrator starts it in team mode (-Dstudio.team=true). Everyone then signs in; the first start creates the user admin with a one-time password shown in the console. Roles: admin (everything, manages users and the licence), tester (everything on projects), viewer (read only). History records who did what. Wrong passwords lock an account for five minutes; sessions end after 30 idle minutes. Passwords are stored only as salted hashes.

To reach Studio from other computers, start it with -Dstudio.bind=<address> and an HTTPS certificate (-Dstudio.tls.keystore=<file.p12> -Dstudio.tls.password=<password>). Studio refuses to listen on a network address without sign-in, and without HTTPS unless you explicitly accept the risk for a trial. Credentials entered for a project are shared by everyone who can run it, so use dedicated test accounts.

Security and privacy

- Studio listens on this computer only and refuses requests from other hosts or other web sites.
- Requests are sent only to the host you name. Production-looking addresses need an extra written confirmation.
- Passwords, tokens and keys are kept out of project files and reports. Evidence is redacted.
- Login tests are capped to protect shared accounts. Create, update and delete APIs are only tested for security unless you opt in.
- The licence is checked offline. No usage data is sent anywhere.
- Use Studio only on systems you are authorised to test.

Licence

Without a licence the product runs as a 30-day trial with up to five projects. Install the licence you received under Licence in the left menu footer. An expired licence pauses running tests and creating projects; reports, history and exports keep working.

Troubleshooting

- All tests are BLOCKED: read the reason in the Blocked chart on Overview. The usual causes are a wrong application URL, a missing credential or an invalid valid-request.
- The check says "This looks like a website address": use Find API address or ask your API team for the base URL.
- 401 on every valid request: set credentials, or the account is locked.
- A test is FAIL but the API is right: open the test, read the expectation and edit it. Use Do not suggest this again to stop similar suggestions.
- The page does not open: start studio.bat again and open http://localhost:8099.

Glossary

- Positive test: a valid request that must succeed.
- Negative test: a valid request changed in one way that the API must reject.
- VAPT: vulnerability assessment and penetration testing report with OWASP and CWE references.
- BOLA: broken object level authorisation, using another user's object.
- Contract: the response shape the specification promises.
- Spec drift: the specification changed after the tests were made.

What is new

23 October 2026

- APIs page shows the real Base URL and API path of every API, explains Tests / Auth / Contract, and Edit address lets you paste one full URL (Studio splits it), set a project-wide base URL and try the API at once.
- A run keeps going when you open another page: a button at the top shows its progress (and an OTP request), a pop-up tells you when it finishes, and Live run shows the last run and earlier runs.
- Group cards show Waiting / Running n of m / Completed.
- Live console and the saved report now look and work like the legacy API Automation console (KPI tiles, progress with time left, execution map, filters, Pause, TV, theme).
- The OTP step is skipped automatically when the login does not ask for an OTP.
- Credentials can be saved on this PC (encrypted) with a Show button, and the Credentials card says when a project signs in by itself.
- Clear old records starts a project fresh.
- Optional AI engine (off by default, local or on another machine): Test ideas page, plain-English failure explanations. Auto-heal repairs after a spec comparison. Export as Karate or Rest Assured code.
- The left menu is grouped in four steps: Get ready, Build tests, Run, Results.

22 October 2026

- Contract checking and spec drift, Data flow (automatic chaining), Business rules to tests, synthetic sample data.
- Load testing, command line and CI templates, scheduled runs.
- SOAP, GraphQL, WebSocket and gRPC support. Mock server. Jira and Azure DevOps defect export.
- Audit trail with integrity check, offline licence, team mode with roles, packaged Windows app.